WSO2

Building Dependable Agents: Where are we?

Srinath Perera , Kaushitha Silva, and Aeshan Rosa

AGENDA & SCHEDULE

Next Three Hours

WSO2

01

Background

02

Agent Basics

03

Hands On Session 1

04

Agent Best Practices

05

Hands On Session 2

06

Building Dependable Agents

Audience Quiz

- How many of you are from Industry?
- How many of you have ever trained a NN?
- How many of you have written code that calls an LLM?
- How many of you have written an AI agent using a framework like LangChain?
- How many of you (or your team) have an AI agent running in Production?

WSO2

SECTION 01

Background

Building Dependable Agents: Where are we?



What will be left for us to work on?

(ICML 2026 keynote)

Arvind Narayanan talk

- AI is a collaborative technology, not an automated technology
  - Reliability challenges:
    - consistency (get it right sometimes vs every time)
    - robustness (work even when conditions are imperfect)
    - calibration (know when you succeeded/failed)
    - operational safety (avoid catastrophic errors)
  - Challenge is verifiability
    - LLM misses some steps, and need to fix them with verification + feedback
    - If we can solve verifiability, we can solve all other problems - e.g. loops do not help much
      - Karpathy: LLMs automate what you can verify
      - Self-reflection becomes context problems [1]

[1] https://people.csail.mit.edu/saman/acpss/talk-2/talk-slides.pdf

Background: What will be left for us to work on? (Contd.)

AI is a collaborative technology

↓

AI is a normal technology.

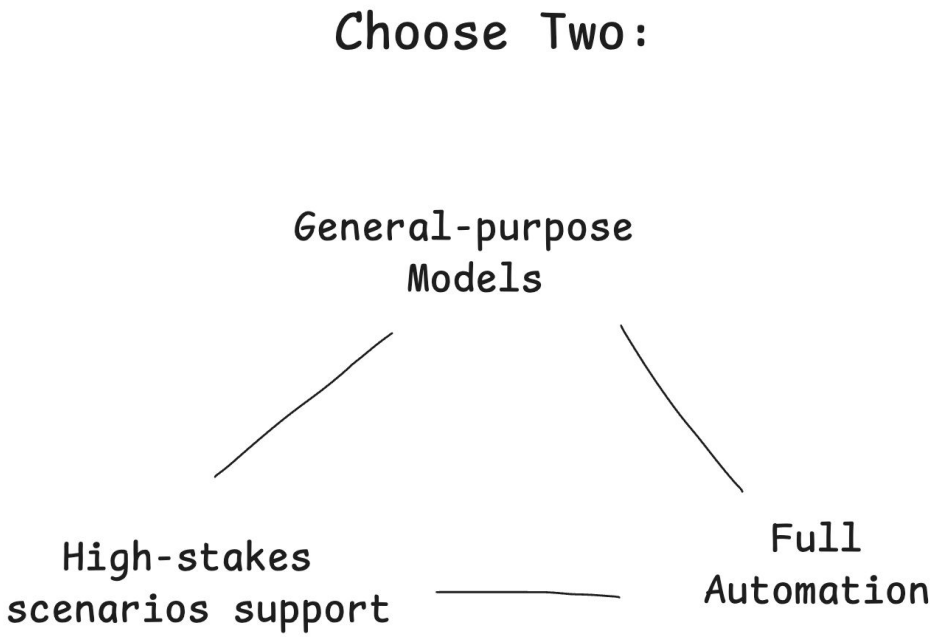
↳ People don't use "electricity" directly.
We use electrical appliances. Those had to be invented,
so that is a kind of downstream innovation.

↓

It takes decades for it to be well integrated



Background: What will be left for us to work on? (Contd.)









Horse and Rider Analogy (from Catholijn Jonker talk)

Now she can move much faster,
but she needs to decide where to go (verification)

Going somewhere blindly will not help,
and could get you into trouble

With a good setup (precautions, harness, training),
a human retains agency, but a horse makes a lot of
decisions too.

Collaborative AI + humans need the same model.

What does that mean?

Until we can completely hand
over problems to AI, the human in the loop
is responsible for the outcome.

- You can delegate work, but not the responsibility.
- Understanding is the key.

Engineers, Researchers, and Scientists seek to understand

- Their value add is "Understanding" and quality guarantee.
  - The worst thing such a professional can do is push for a solution they do not understand (within reason)



History: Initial ideas

- We first tried to build intelligent systems using rules.
  - While it can capture common cases, it's hard to capture most cases - e.g., rules to learn to drive.
- This is not how humans can learn.
  - Switch to learning from data.
  - Regression is a good way to think about it.
  - Optimization problem (hill climbing), work well for linear problems
  - Nonlinear data: transform to linear space and do regression
- Other algorithms
  - Random Forest, XGB ..
  - Neural Networks
  - Back propagation
- Deep neural networks
  - Learn intermediate representations

History: Rise of GPUs

- We did not have enough computational power to train large deep NNs
- Datasets
  - ImageNet, etc
  - GPUs let us do backpropagation efficiently.
- NNs start to beat all other solutions.
  - 2012 - ImageNet (AlexNet, 11pp improvement)
  - 2015 -ResNet surpassed the human error threshold
  - AlphaGo (2016) & AlphaZero (2017)
- NN variations
  - MLP, RNN/ LSTM, CNN, RL

History: Self-Supervised Learning and Embeddings

- Labeled data is scarce; unlabeled data is abundant
  - Self-supervised learning hides part of the data and trains the model to predict it
- The model learns a representation (embedding) as a by-product
  - Text: predict context or next/masked token (word2vec, BERT, GPT)
  - Images: reconstruct or predict masked regions (autoencoders, JEPA)
- Pre-train self-supervised, then fine-tune on a task → good results with little labeled data

History: Can we reuse weights?

- Pre-2017: AI relied on narrow, specialized deep learning models (like Word2Vec) trained on single tasks.
- 2017: Google researchers published the "Attention Is All You Need" paper, introducing the Transformer architecture and self-attention mechanisms that made parallel, large-scale training possible.
- 2018 (BERT): Google released BERT, a bidirectional transformer model that set new records in language understanding.
- 2019 (GPT-2): OpenAI demonstrated powerful text generation with GPT-2.
- 2020 (GPT-3): In context learning
- 2022 (ChatGPT): pre-training + RLHF.
- 2023 (GPT-4) Frontier Reasoning & Long Context

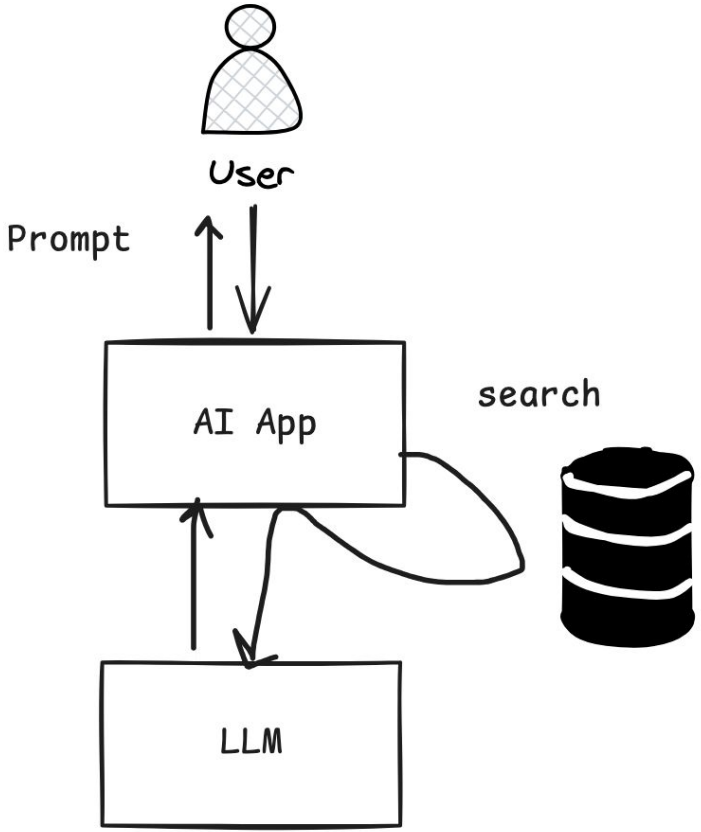
LLMs

- Pre training
  - Self-supervised learning to predict text
- RLHF
  - Fine-tune LLMs using Human feedback
- Scaling laws and Emergent behaviors
  - Zero-Shot and Few-Shot Learning
  - Chain of thoughts / Reasoning
  - Hallucinations
- Post-training, GRPO, DeepSeek
- Inference time scaling

Can I extend the LLM?

- New data, new skills?
- Option 1:
  - RAG + Tools
- Option 2:
  - Fine tuning
- You can do both, too
- New options are memory and skills (discussed later)

Option 1: RAG Architecture + Tools



- Search can happen in many ways
  - From DB
  - From Inverted Index (e.g., Lucene)
  - Approximate Nearest Neighbor (ANN) search
    - Hierarchical Navigable Small World (HNSW)
    - IVF-PQ (Inverted File with Product Quantization)
    - Locality-Sensitive Hashing (LSH)
  - From APIs
- Current best practice is to construct context much more deliberately than RAG architecture with verification, tool calling, etc. (Context Engineering)

Option 1: RAG Architecture + Tools (Contd.)

- Tools are helper functions for Agents, and they provide data, specific logic, and APIs that will help the LLM.
  - We include the available tool list (definitions only) in the context of the LLM call
  - LLM replies with which tool to execute
  - Agent will execute the tool, include results in the context, and call the LLM, which may suggest the next tool
- MCP is standard for Agents to discover available tools
- Modern LLMs are specifically fine-tuned to use tools well.
- Thoughtfully composing tools is a good way to control what the LLM does
  - E.g., provide deterministic code as tools
  - E.g. LiveGraph(2026) as an example

Option 2: Fine Tuning

- SFT
  - Use back propagation to adjust output distribution
- GRPO
  - Use RL
  - LLM will give multiple solutions, and we need to give rewards to those solutions based on relative merit

Background: RL

- Learn by trying
  - Learn best policy at given state
  - Learn from feedback
- Developments
  - Q-learning (1989), REINFORCE (1992)
  - MCTS (2006)
  - 2013–Deep Q-Networks (DQN)
  - 2016–AlphaGo
  - 2017–Proximal Policy Optimization (PPO)
  - 2017–AlphaZero
  - 2022–Train LLMs with RLHF
  - 2024–GRPO

Option 2: Fine Tuning is the last resort

- Training is expensive**, complex, and unpredictable
  - Getting RL correct need expertise e.g. Catastrophic Forgetting & Distribution Shift
- Hard to update with new data - Need periodic retraining with new data
- Serving and update pipelines are much more complex
- Switching to new models needs retraining
- Multi-tenancy and security is hard
- Harder to trace

** if fine tuned model is reused a lot, cost may be justifiable

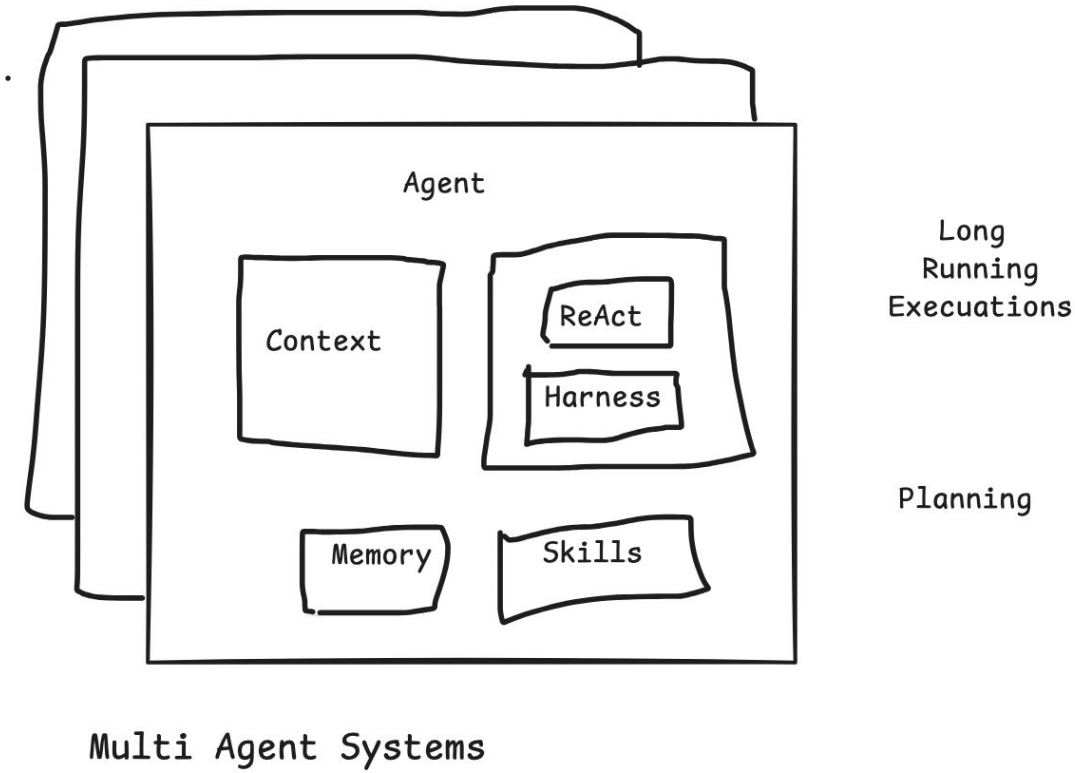
WSO2

SECTION 02

Agent Basics

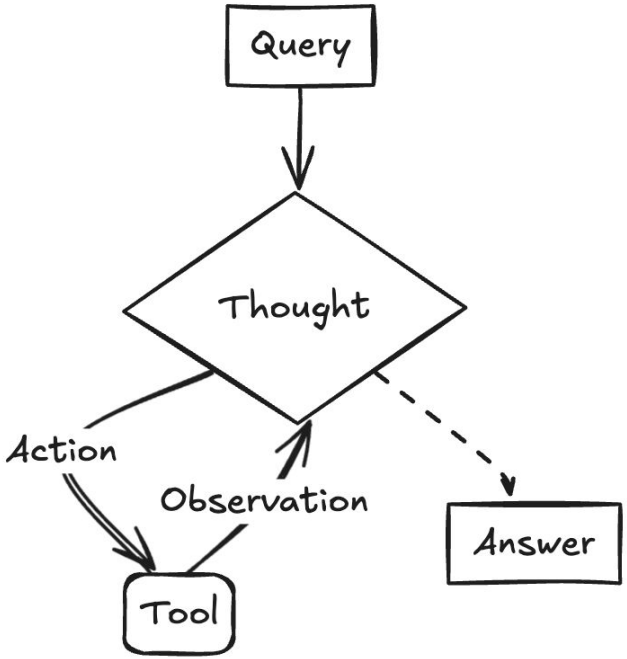
Building Dependable Agents: Where are we?

Agent Design Concepts



Reasoning and **Act**ing **agents (ReAct agents)**

- Instead of planning and executing, ReAct runs a loop.
- On each iteration
  - Reason the next action, carry it out, and add results to context.
  - Compared to plan-and-run, ReAct evaluates step by step and is very effective.
- De facto pattern for most Agents
- With a recovery log, it can beat complex planning (e.g., RAC (2026)) on dynamic problems.
- When context is large, ReAct can be expensive as each iteration is expensive (e.g., large document analysis-related tasks)



RAC (2026), https://dl.acm.org/doi/abs/10.1145/3786335.3813141

Agents

- Agents are “*codeword*” for carefully engineered solutions
- An Agent
  - Has a goal and tools
  - Interacts with tools and the environment and decides when to stop.
  - Often includes verification
  - Individual Agents often use ReAct.
- Multiple Agents can be combined as an Agent Graph.
  - Connections can be conditional and have retry logic.
  - Can be used to combine AI and non-Agentic logic
- We implement Agent graphs using a framework.
  - LangChain , CrewAI, DSPy, Microsoft AutoGen

Multi Agent Systems

- Multi-Agentic solutions break the problem into multiple roles and assign them to different Agents.
  - E.g., code generation (AgentCoder-2023) has a programmer Agent (Agent #1), a test designer Agent (Agent #2), and a test executor Agent (Agent #3)
- Five canonical architectures: (from Towards a Science of Scaling Agent Systems, https://arxiv.org/pdf/2512.08296)
  - Single Agent System (SAS)
  - Four Multi-Agent variants by how they coordinate
    - Independent
    - Centralized
    - Decentralized
    - Hybrid

Multi Agent Systems Best Practices

- Workflows vs Agents (autonomy as a dial). Many tasks should be deterministic workflows with LLM steps, not open-ended Agents.
  - From Anthropic "Building Effective Agents" taxonomy (prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer) and "levels of autonomy" framing.
- Most common reason to use multiple Agents is to manage the context
  - Most systems use a centralized model with multiple Agents to manage the context and for separation of concerns
- Real emergence is hard and requires careful work; it should not be used lightly.

Context (Engineering)

- Information you give to LLM (e.g. Retrieval-Augmented Generation)
  - RAG is example, but you can be more selective
- Context Problems Ranked
  - Incorrect information
  - Missing information
  - Too much information
- Solutions
  - You only have approximately 170k of context window to work with.Use about 40% of the context, after that more you use the context window, the worse the outcomes you'll get.
  - Context compaction. (see [1])
  - Progressive disclosure (include summary, give tools to extract more)
  - Sub-Agents
    - not for “role play” they are for context management,
    - find as much information as possible, plan, and then hand over to sub-Agents
  - Good: [1] https://people.csail.mit.edu/saman/acpss/talk-2/talk-slides.pdf

Memory: How it works (Developer Experience)?

LLMs can’t remember things; memory solutions add memory. You can also see it as an extension of context.

- Pattern 1: Automatic injection (most common)
  - The orchestration layer — not the LLM — queries memory and prepends results to the prompt before the model ever sees it. (MemoryBank, ENGRAM, RMM, ExpeL, A-MEM, and MAGMA all work this way.)
- Pattern 2: Tool call (Agent-directed retrieval)
  - The LLM itself decides when to query memory and what to search for, by generating a structured function call. MemGPT
- Pattern 3: Internal state
  - No retrieval at all (e.g. via RL) - e.g. MEM1

Memory: Storage & Retrieval

- Storage
  - Knowledge graph
    - Temporal - Zep (2025)
    - Keep multiple graphs (temporal, similarity, casual, co-occurring) - MAGMA
  - By resolution: Individual utterances, full turns, and whole sessions
  - By type:
    - episodic (specific events — "on Tuesday, the user asked about Python")
    - semantic (general facts — "the user prefers Python over Java")
    - procedural (how-to knowledge — "when debugging, always check logs first").
  - Separate short term scratch pad (go to context always), vs. write key point to disk (like we do with claude vs. detailed storage (e.g. knowledge graph)
- Retrieval
  - More prominence based on recent/ last access - MemoryBank (2023)

Skills

- Reusable instructions or procedures
  - e.g., if you want to control how Agent will check if source is relevant to a topic , you can write a skill
  - verifications, constraints, tools or sources, steps
- Prompt can directly refer to skills or Agent may implicitly use appropriate skills
- Almost all Agent frameworks support skills
- Automatic reflection loops may add new skills

Harness

- Term has gotten a lot of popularity lately, and sometimes add 10+ pp.
- A harness includes:
  - Planning + execution (e.g., Ralph Wiggum technique - each run improves, run tests, write to progress.txt, and die)
  - System prompts, CLAUDE.md, AGENTS.md, skill files, and subagent prompts
  - Tools, skills, MCP servers, and their descriptions
  - Bundled infrastructure (filesystem, sandbox, browser)
  - Orchestration logic (subagent spawning, handoffs, model routing)
  - Hooks and middleware for deterministic execution (compaction, continuation, lint checks)
  - Observability (logs, traces, cost and latency metering)
- Every mistake can becomes a rule (skills)
- Reading
  - https://martinfowler.com/articles/exploring-gen-ai/harness-engineering-memo.html
  - https://martinfowler.com/articles/harness-engineering.html
  - Birgitta Böckeler - State of Play, https://www.youtube.com/watch?v=tFffUnSq7VA

LLM Reasoning is Fallible

- LLM decision quality are highly dependent on
  - Instructions
  - Context
  - Steps in detailed instructions, (e.g., around 20 or so steps, the LLM starts to hallucinate and starts missing or misrepresenting steps)
  - Feedback
  - Complexity of the task
- Often ReAct works better than Zero-Shot.
- Harnesses often work better than vanilla ReAct.
- Sometimes asking to plan helps; sometimes it hurts.
- Asking to reflect on its output often helps.
- External Feedback and validation help.
  - E.g., compiler feedback, constraint checks, feedback from environments

Improving Reasoning with Feedback

- Grounding vs. Hallucinations
  - LLM as Judge
- Compilation and Execution feedback for code
- Public/private test feedback
- Simulations
- Inference time scaling
  - Exploration vs. exploitation using MCTS
- Generating verifiable code
  - e.g., using formally verifiable programming language like Dafny

Advanced Reasoning Approaches

- Multi-Agent Debate
  - AgentCoder, PlanGEN (2025), COMPASS(2026), Shepherd (2026)
- Reflective Reasoning
  - ADaPT (try ReAct, then reason about trace) , Self-Refine: Iteratively critiques and revises its own output (Madaan et al., 2023), CRITIC: Uses external tools to validate and repair outputs (Gou et al., 2024).
  - LLM-Modulo: Generates candidate plans and uses external critics to check constraints and request revisions (Kambhampati et al., 2024).
- Memory based reflections
  - CRISP(2026) - self-distilling
- Tree/ MCTS
  - LATS: Explores alternative trajectories using tree search, environmental feedback, and backtracking (Zhou et al., 2024).
  - Tree of Thoughts (2023) - build a tree from thoughts (how to solve the problem), use an LLM as a judge, and then use BFS, DFS, etc to find the best solution
  - Reasoning with Language Model is Planning with World Model (2023) - MCTS + feedback from LLM.
  - Planu (2026) MCTS
  - Memory + MCTS - BREW (2025) - distill past traces + MCTS
  - Multi-Agent + MCTS - From Assumptions to Actions(2026) - MCTS

Long Running Executions

- Challenges
  - Finite context.
  - No persistent state
  - Error compounding (No self-verification) .
- Handled via
  - Ralph Wiggum technique (Many harnesses support this)
  - Careful verification loops
- Reading
  - https://addyosmani.com/blog/long-running-agents/
  - https://www.mindstudio.ai/blog/how-to-build-long-running-ai-agent-7-components

WSO2

SECTION 03

Hands-on Session

Building Dependable Agents: Where are we?

Hands-On Session

- Code is in https://github.com/kaushithamsilva/icter-workshop
  - Follow the ReadMe.md
  - Follow Hands on Session 01

WSO2

SECTION 04

Agent Best Practices

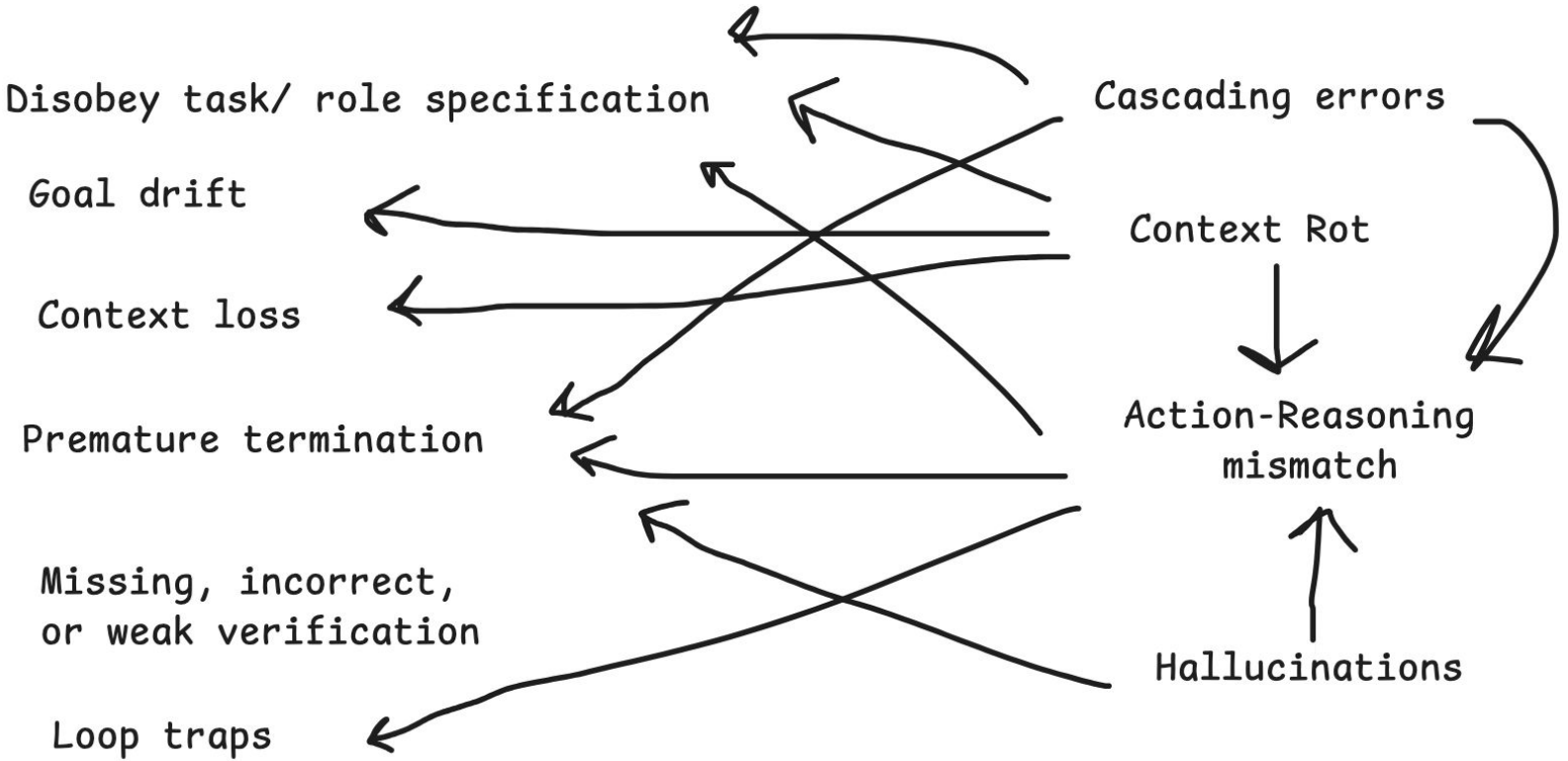
Building Dependable Agents: Where are we?

Common Failure Patterns (Symptoms)

- From Cemri, Mert, et al. "Why do multi-agent llm systems fail?." Advances in Neural Information Processing Systems 38 (2025).
  - Disobey task specification, Disobey role specification
  - Goal drift — The Agent loses track of the original objective and optimizes for something adjacent
  - Loop traps — The Agent keeps retrying a failed action without any mechanism to recognize it's stuck
  - Context loss — After many steps, the Agent "forgets" earlier decisions or constraints
  - Premature termination
  - Missing, incorrect, or weak verification

** There are other studies too, above is the most recent.

Looking beyond Symptoms: Common Root Causes



Evaluation

- Problems
  - LLM-based apps are probabilistic.
  - Answers are in natural language; validation is hard. (Often use an LLM as a Judge with human calibrations for this case)
  - Even small changes (e.g., prompt changes) can significantly change outcomes.
  - Can’t trust gut feel - Can’t trust a few successes and assume the app works.

Evaluation (Solutions)

- Need a benchmark with ground truths.
- Start evals early and small, built from real usage.
  - e.g., Test multi-turn behavior with a simulated user
  - Use an LLM to generate hard test inputs, and treat the acceptable pass rate as a product decision.
- Consider starting from well-known benchmarks and adapting (read how related benchmarks are created; it will give you ideas)
- Look out for data leaks (if you design or parameter-tune looking at a dataset, it is not a good benchmark)
- Align LLM judges with human labels and keep re-checking them.
- Reliability metrics for Agents:
  - Pass@K - try k times, pass if at least one is correct
  - Pass^k - try k times, pass only if all pass
  - Cost-normalized accuracy (Cost per success)
    - Kapoor et al. "AI Agents That Matter"
- Periodically test against the benchmark (part of the release train).
- Need to cross-check against the minimum bar; some use cases may not be feasible.

If you are Publishing a Research Paper

- Select and justify benchmarks.
- Try not to create your own benchmarks.
- Test against 2–3 recent baselines.
- Repeat LLM-related tests at least 3 times and report the standard deviation
- Test across 2–3 different LLMs
- Do ablations

Best Practices: Step up complexity ladder deliberately

- Start simple and add complexity only when it measurably helps
  - Deterministic workflows
  - Deterministic steps with LLM calls
  - ReAct
  - Centralized/orchestrator-workers
  - Decentralized Coordinating Agents
- Use subagents for context isolation, not role-play
- RAG as tools
- Harness with logic provided via the prompt
- Graph with harness at each node

Best Practices

- Prototype with the most capable model, then swap down step by step.
- LLM Agents are much more sensitive to bad error messages, descriptions, etc
  - Write actionable errors and mistake-proof parameters ("poka-yoke").
  - Use unambiguous names (user_id, not user).
- Reflect at key checkpoints
- Define explicit stopping conditions and limits. Use max iterations, turn limits, and clear exit conditions. Verify against the real objective, end to end, as a user would.
- Detect and handle when queries deviate from training data.
- Keep the tool set small and unambiguous - run ablations and remove
- The model fills gaps in the problem with its most likely default.
- Tune for cost and latency.
  - Prompt caching - KV-cache-friendly prompt layout (stable prefixes, append-only context),
  - Model routing - small-model routing/cascades, budgets per task.
  - Cost runaways are both a reliability and a security (denial-of-wallet) issue.

WSO2

SECTION 05

Hands-on Session 02

Building Dependable Agents: Where are we?

Hands-On Session

- Code is in https://github.com/kaushithamsilva/icter-workshop
  - Follow the ReadMe.md
  - Follow Hands on Session 02

Building Dependable Agents

Risk/ Challenges

- Shadow AI / no inventory / Unclear ownership and approval
- Quality of outcomes (Agent does main task well)
  - Ensure quality
  - Sustain - Drift, quality and hallucination (Models degrade after deployment and LLMs produce wrong answers)
- Third-party and foundation model risk
- Risks arise from LLM/ Agents operations (can’t design them out)
- Regulatory compliance and audit evidence
  - EU AI Act, ISO 42001, NIST RMF or sector rules

Risk Categorization

Based on OWASP Top 10 for Agentic Applications 2026 (ASI) and the OWASP Top 10 for LLM Applications (LLM)

- LLM06:2025 Excessive Agency (do more than what it allowed to do)
  - ASI01: Agent Goal Hijack
  - ASI02: Tool Misuse and Exploitation
  - ASI03 (Identity & Privilege Abuse)
  - ASI05: Unexpected Code Execution (RCE)
- ASI04: Agentic Supply Chain Vulnerabilities / LLM03:2025 Supply Chain
  - (backdoors in LLMs, data, tools)
- ASI06: Memory & Context Poisoning / LLM04: 2025 Data and Model Poisoning
- Agent specific
  - ASI07: Insecure Inter-Agent Communication
  - ASI08: Cascading Failures (system wide harm)
  - ASI09: Human-Agent Trust Exploitation
  - ASI10: Rogue Agents
- LLM01:2025 Prompt Injection
- LLM02:2025 Sensitive Information Disclosure
- LLM05:2025 Improper Output Handling (output validation)
- LLM07:2025 System Prompt Leakage (detail about how it works leaks)
- LLM08:2025 Vector and Embedding Weaknesses (users use that to inject harmful content)
- LLM09:2025 Misinformation
- LLM10:2025 Unbounded Consumption

How to think about this?

- Even without an attacker, LLM/ Agent can do bad things: (e.g. Hugging Face Attack)
  - Miss things, hallucinate, take wrong actions, and do harm via bias, hate speech, etc.
  - Expose protected data.
- Attackers may get Agents' LLMs to do the same.
- Any input (prompts, memory, context, tool response, retrieved content, tool output, tool descriptions, memory and messages from other Agents, etc.) from any path can have injection.
- Any outbound channel can be an exfiltration path. (e.g., results, rendering, HTTP, side-effecting tools, the user's clicks, and trusted domains that can be abused)
- Prompts and LLM-based decisions do not fully remove the risk.
- There are no magic solutions; we need to engineer them to be safe.

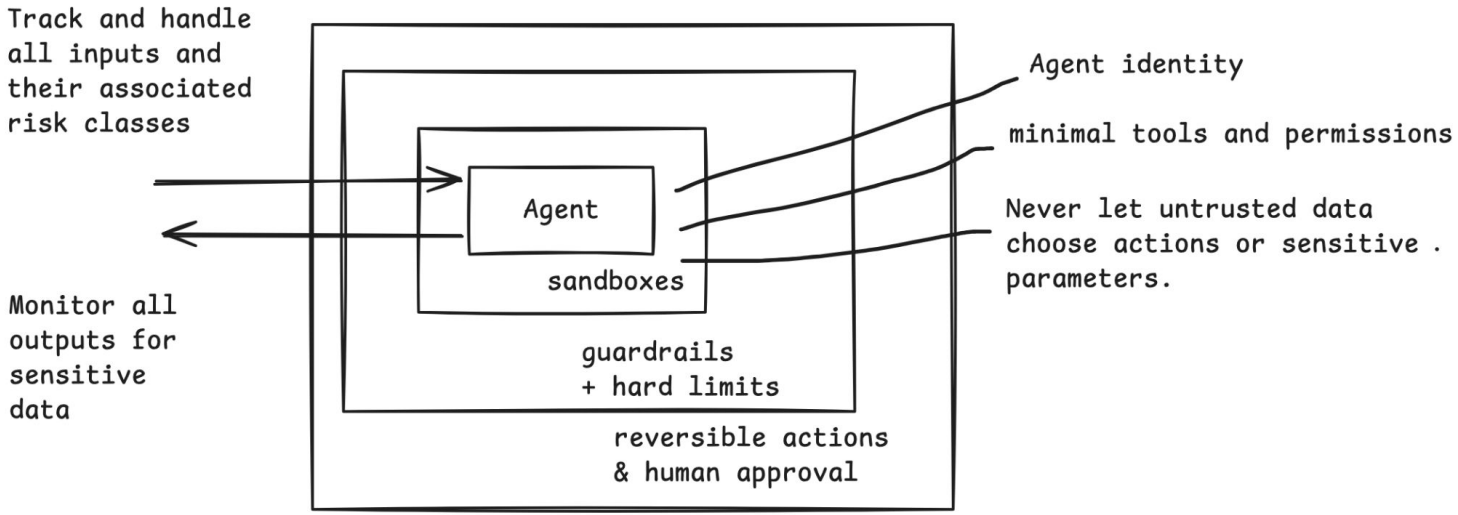
Principles for Building Dependable Agents

- Four key principles (OWASP Software Security Design Principles) are
  - The principle of Zero Trust
  - The principle of Least Privilege and Separation of Duties
  - The principle of defence-in-Depth
  - The principle of Security-in-the-Open
- Eliminate all exposure to catastrophic tail risks
  - “eliminate all exposure to catastrophic tail risk while aggressively seeking bounded, small-scale risks” (“adapted from Taleb’s Antifragile”).

Principles in Practice

- Zero trust
  - Agent identity
  - Never let untrusted data choose actions or sensitive parameters.
  - Track and handle all inputs and their associated risk classes
- Least Privilege and Separation of Duties
  - We should define tools and permissions tightly to control risk.
  - Break the lethal trifecta. No single context should combine private data, untrusted content, and an exfiltration channel.
- Defence in depth and eliminate all exposure to catastrophic tail risks.
  - Use sandboxes, guardrails, hard limits (e.g., a budget for Agents, tokens, cost, or time), and human approval as defence in depth.
  - Human Approval
  - Use tamper-evident logs, anomaly detection, kill switches, and incident response.
  - Post hoc audits identify weaknesses and fix

Building Dependable Agents





Post hoc audits identify weaknesses and fix

Use tamper-evident logs, anomaly detection, kill switches, and incident response.

Risk Handling in Agent Development Lifecycle

- Testing
  - Unit tests and Benchmarks as pre-deployment checks
  - Red teaming
    - evaluation that checks Agents against business KPIs; judging models from a different vendor than the model being tested; and reliability metrics such as pass^k
    - Threat modeling with MITRE ATLAS and the OWASP Agentic Top 10
    - E.g., PyRIT (Python Risk Identification Tool) is Microsoft AI Red Team's framework for automating attacks on generative AI systems.
- Monitoring
  - Continuous monitoring to detect if results are degrading
  - E.g., Langfuse is a self-hostable LLM engineering platform that captures traces, runs online evaluations, and tracks quality, cost, and latency over time.
- Post hoc analysis for logs, identify problems, adjust guardrails based on findings
  - Need Custom solutions using ideas like Meta Harness.

Zero Trust, Least Privilege

- Agent identity, authentication
  - Separate agent permissions
    - Platform permissions
    - Permissions from the current user it's acting on behalf of
      - Agents acting on behalf of users need scoped, short-lived, delegated credentials (OAuth 2.x token exchange, MCP authorization spec, per-tool scopes, audit attribution).
  - At Audit, differentiate agent actions on behalf of a user and user actions.
- We should define tools and permissions tightly to control risk. Give the minimum capability required by the Agent to carry out the task.
  - Avoid board MCP or shel without restrictions; instead, provide access within a sandbox, following least privilege and Separation of Duties.

Zero Trust, Least Privilege (Contd.)

- Hierarchical budgets per key, team, or Agent that stop requests on the request path; auto-pause; rate limits
  - AI Gateways
  - LiteLLM, Bifrost (Apache 2.0; hierarchical budgets per virtual key, team and customer), Paperclip (budgets per Agent and team, auto-pause)
- Never let untrusted data choose actions or sensitive parameters.
  - Track where each value came from
  - Attach provenance and capability metadata to every value (where it came from, who may read it), and track data flow between values.
- Treat model errors and harmful content as their own risk class. Control them with grounding, evaluation, and content safety rather than security controls.

Defense in Depth and eliminate all exposure to Catastrophic Tail Risks

- Guardrails (more later)
  - All inputs and output paths
- Human Approval (more later)
  - Require approval for irreversible actions and make rollback possible.
  - Document residual risk and catastrophic risks, and add approvals
- Use tamper-evident logs, anomaly detection, and post hoc audits (more later)
- Hard limits (e.g., a budget for Agents, tokens, cost, or time)

Defense in Depth and eliminate all exposure to Catastrophic Tail Risks (Contd.)

- Sandboxes
  - Sandboxed execution in microVMs or execution rings, with credentials and network access controlled from outside the sandbox
  - E.g., Perplexity SPACE (Apache 2.0; Firecracker; credentials and network held outside the sandbox), E2B (Apache 2.0; Firecracker; self-hostable), Building blocks: Firecracker, gVisor, Kata Containers, Microsoft AGT Agent Runtime (execution rings)
- Don't Break the lethal trifecta
  - No single context should combine private data, untrusted content, and an exfiltration channel. If a task needs all three, split it across isolated components or require human approval. (Simon Willison, https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/)
- Agent kill Switch
  - E.g., Microsoft AGT (kill switch, circuit breakers, SLOs), Paperclip (approval gates, budgets, auto-pause, goal hierarchy), LangGraph interrupt() (MIT), Bernstein (deterministic orchestration and replay), Galileo Agent Control (Apache 2.0)

Defence in depth > Guardrails

- What to check for (allow vs. deny models)
  - Prompt Injection Defense, Risky Operations, PII and sensitive data, Compliance checks, Custom Policies (application-level checks) , SQL injection and blocklist checks on tool arguments, Fact-checking, Jailbreak detection, Contextual grounding, Harm and fairness
- Where
  - Input Validation
  - Memory & Context Security
  - Output Validation & Guardrails
  - Tool calls
- Ideas
  - Dual LLM (Privileged vs. Quarantined) - two LLMs for trusted and untrusted processing
  - CaMeL (Capabilities and Information-Flow Tracking) tracks where information flows and blocks risky actions.

Defence in depth > Guardrails (Contd.)

- Implementations
  - Policy as code
  - Policy as Rule
  - Policy via LLM
- Tools
  - APIM/ AI gateway policies
  - Some checks can be enforced as a firewall or sandbox
  - Some run alongside the Agent e.g. NeMo Guardrails (Apache 2.0), Llama Guard

Defence in depth > Reversible Operations and Human Approval

- Reversible operations can defend against many risks, since systems can audit and fix mistakes. However, reversing operations also has costs, and those need to be considered.
- Using human approval for risky (e.g., financial, administrative, or externally visible) irreversible actions is an effective defence.
- The right level of human involvement is complicated.
  - If humans know AI is checking, that may affect outcomes.
  - Giving human input may bias the AI.
  - Asking for a third opinion when both disagree may be effective, but expensive.
- Need to make sure
  - Humans have full context in an understandable form.
  - Alert and Review Fatigue ("Rubber-Stamping") does not happen (second driver to make sure driver is not sleeping)
  - Not in "Moral Crumple Zone": Sociotechnical systems often use human verification as a liability shield.
- Watch for a False Sense of Regulatory Safety.
- The medical field is a good use case to follow.

Defence in depth > Audit, Monitoring, Improvements

- Observability and tracing for Agents.
  - Collect the data with correct resolutions
- Audit, Monitoring & Observability
  - Check for known (Audit, compliance, known problems) and unknown risks (E.g., Anomaly Detection for drift detection)
- Improve the system based on findings
  - Include new skills
  - Adjust guardrails
  - Fine tune parameters
- Above cycle can be both offline and online

Defence in depth

- The key challenge in defense-in-depth is the tension between false positives and coverage (precision and recall)
  - Too many false positives mean people lose trust
  - Not enough coverage exposes risk
- Need to monitor and calibrate this.

What We Discussed!!

- Agents are code for carefully engineered solutions
- Ground your Agents with Evaluations
- Best practices: Step up complexity ladder deliberately
- Even without an attacker, LLM/ Agent can do bad things
  - Understand trusted and untrusted parts of the system and design carefully
- Build on four key principles (OWASP Software Security Design Principles)
- Defense in depth is a key idea
- Audit and improve your defenses

Questions?